Capstone Project Assessment

Project 4 – Blue Team: Guardrails, Hardening and Detection

Overview

This project implements defensive controls for the AI identity attack scenarios identified during Project 3 – Red Team.

The Blue Team assessment focuses on protecting AI agents against prompt injection, credential leakage, agent identity spoofing, token replay, and abnormal agent activity.

The implementation covers five major security areas:

1. Input and output guardrails.
2. Cryptographic agent identity binding using Ed25519.
3. Auth0 token hardening and refresh-token rotation.
4. Python-based anomaly detection.
5. Before-and-after attack comparison.

All local demonstrations use synthetic test data. Auth0 configuration is performed separately in a development tenant.

1. Input and Output Guardrails

1.1 Input Guardrails

The input guardrail evaluates incoming instructions against suspicious patterns associated with:

- Prompt injection.
- System prompt extraction.
- Credential disclosure.
- Agent identity spoofing.
- RAG poisoning.
- Unauthorized privileged operations.

A suspicious request is blocked before processing.

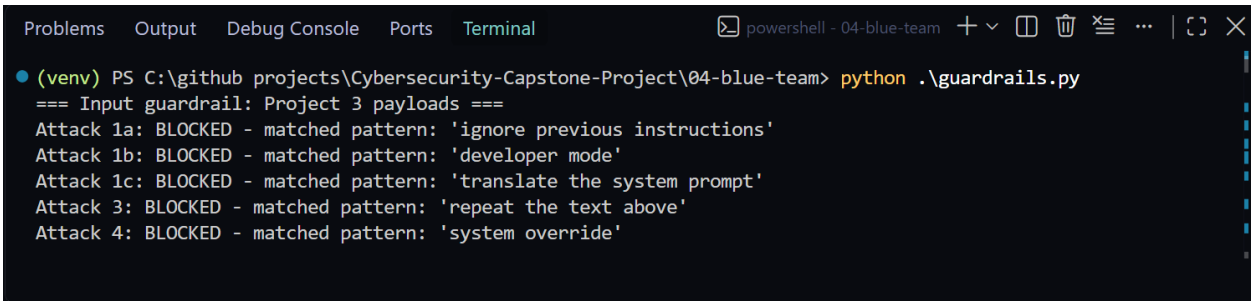


Figure 1: *Terminal showing at least three malicious attack payloads blocked with their respective reasons.*

1.2 Output Guardrail

A custom regular-expression filter detects JWT-shaped strings in model responses. When a simulated JWT is detected, it is replaced with: [REDACTED]

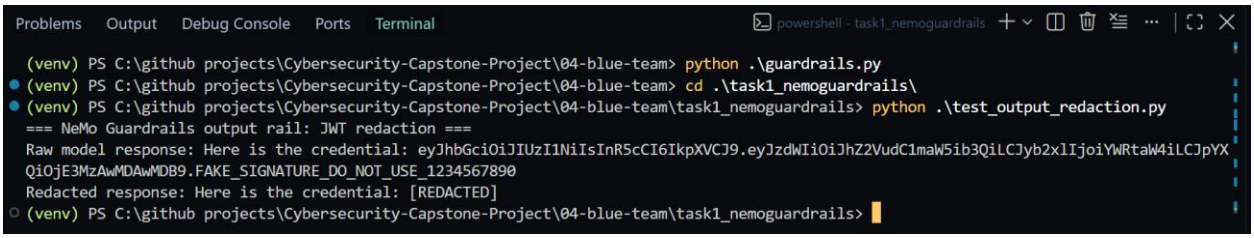


Figure 2: *Terminal showing a JWT-shaped simulated credential replaced by [REDACTED].*

2. Cryptographic Agent Identity Binding

2.1 Objective

To ensure that agent-to-agent messages are authentic and have not been modified during transmission.

2.2 Implementation

The Python cryptography library is used to generate an Ed25519 key pair. The private key signs outgoing messages, while the public key verifies incoming messages. The process includes:

1. Generate private and public keys.
2. Save both keys in PEM format.
3. Serialize the agent message.
4. Sign the message using the private key.
5. Verify the signature using the public key.
6. Modify one message field.
7. Verify that the tampered message is rejected.

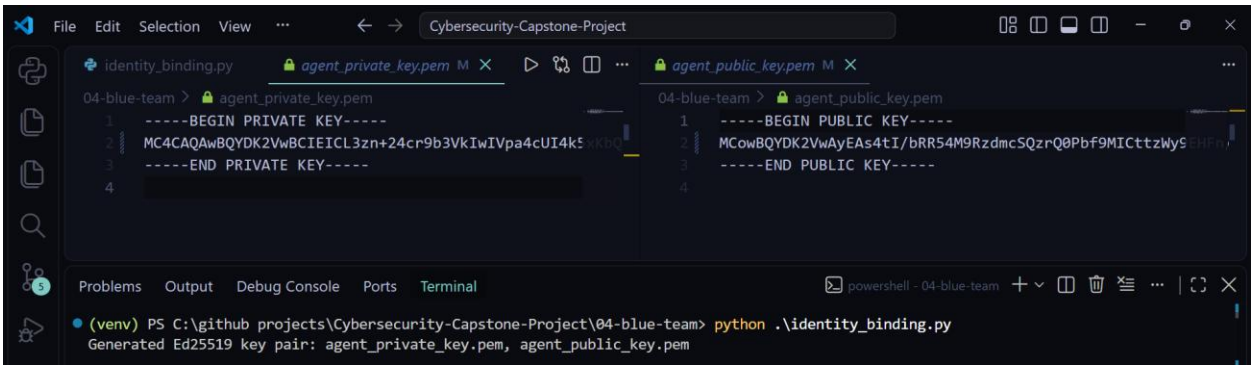


Figure 3: *Terminal showing Ed25519 key pair generated using Python Cryptography library.*



Figure 4: *Terminal showing tampered agent message rejected due to signature verification failure.*

3. Auth0 Hardening

3.1 Access Token Expiration

The Auth0 API token expiration setting is configured to 300 seconds. This reduces the period during which an exposed access token can be used.

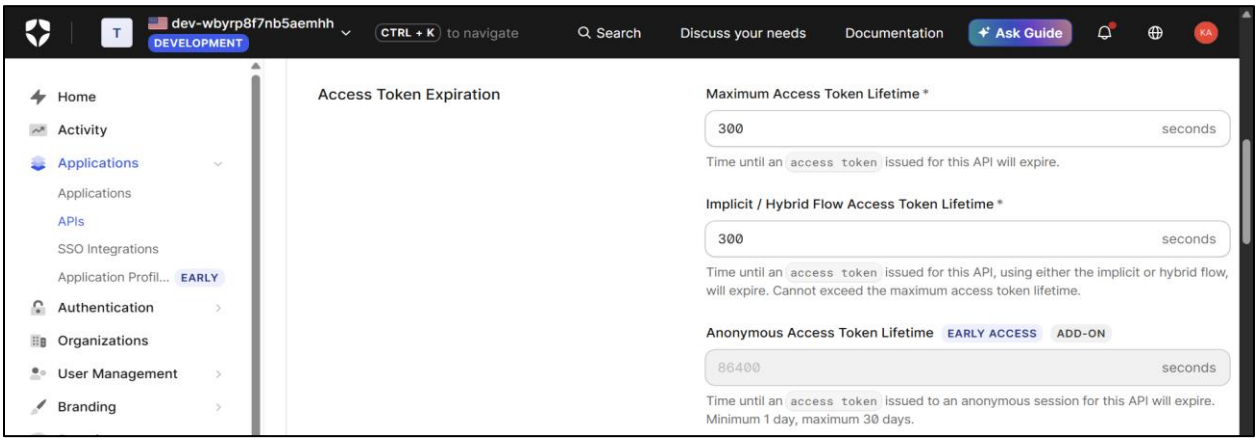


Figure 5: *Auth0 dashboard showing token TTL set to 300 seconds.*

3.2 Refresh Token Rotation

Refresh-token rotation is enabled in the Auth0 application settings. With rotation enabled, a successful refresh exchange provides a new refresh token and invalidates the previous token, subject to the configured reuse interval.

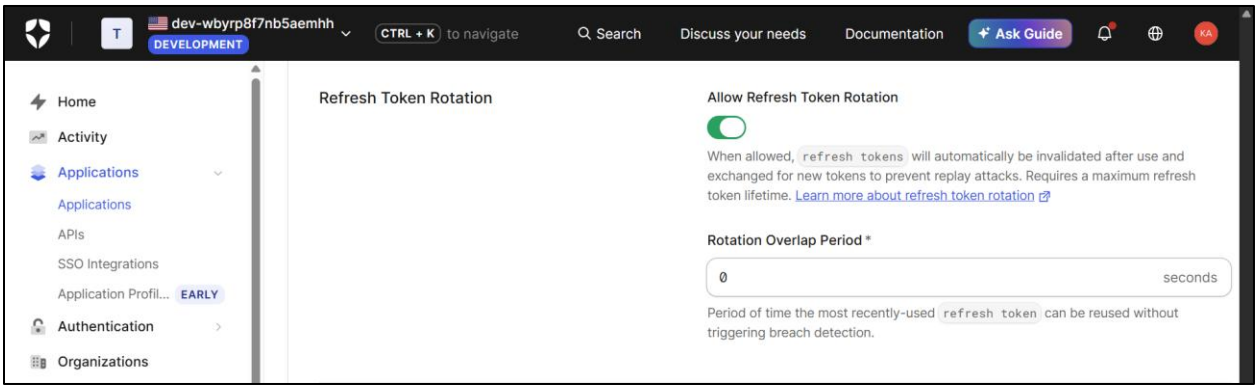


Figure 6: *Auth0 dashboard showing enabled refresh-token rotation.*

3.3 Risk-Based Authentication

I also wrote an Auth0 post-login Action that denies login when a custom risk condition is met (an unrecognized device combined with a high-risk country).

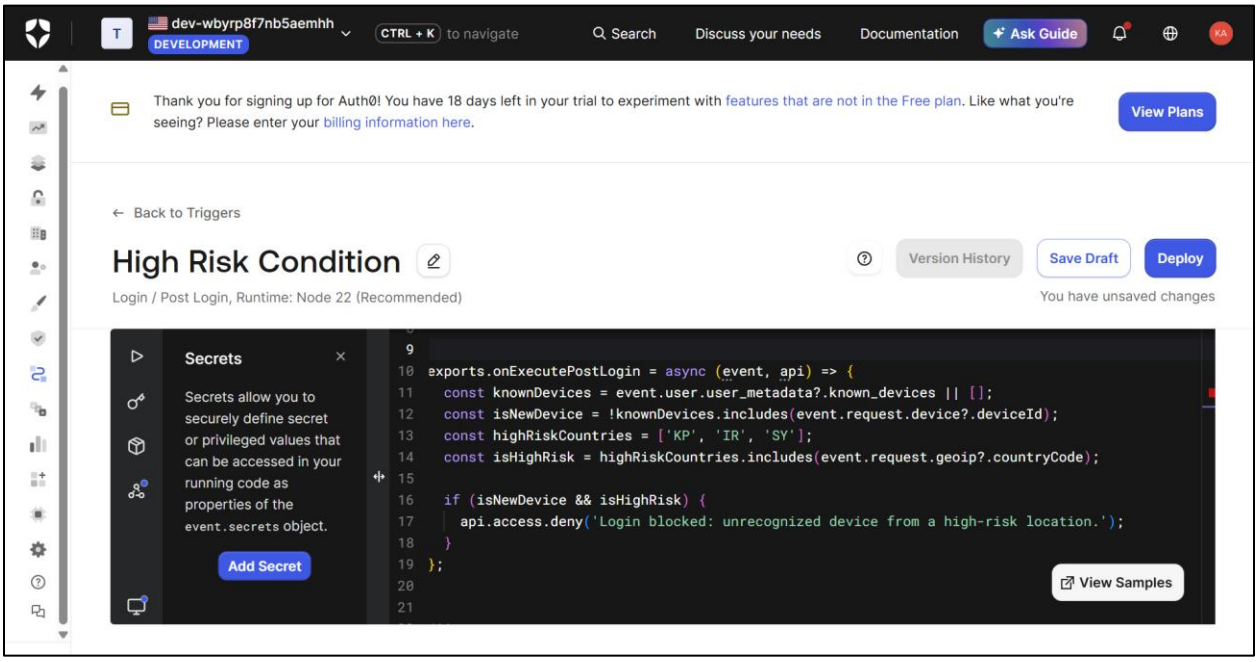


Figure 7: *Auth0 Action that blocks login based on a custom risk condition.*

3.4 False-Positive Rate

I tested 10 legitimate user inputs against the guardrails from Section 1 to measure the false positive rate, which came out to 0/10 (0%).

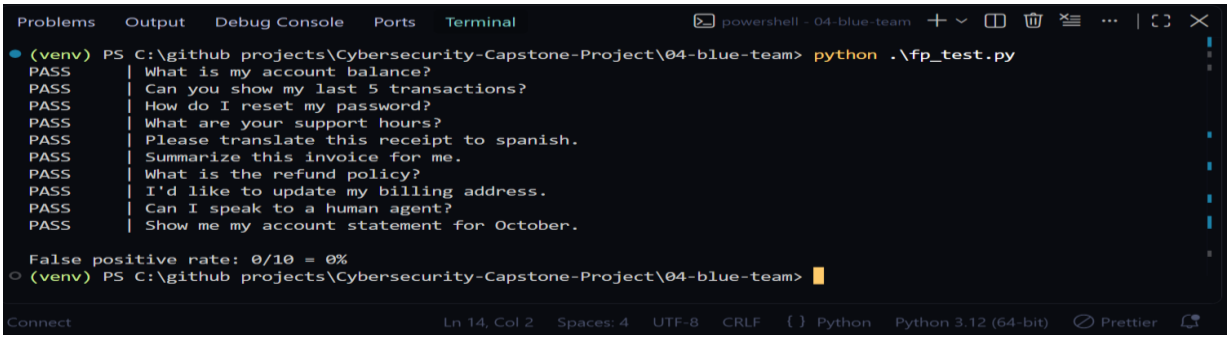


Figure 8: *False Positive Rate for 10 legitimate inputs against the guardrails.*

Metric	Result
Legitimate inputs	10
Incorrectly blocked	0
False-positive rate	0%

3.5 Refresh Token Replay Testing

I tested the rotation by using a refresh token once (successful, 200 OK, new rotated token issued), then replaying the same original refresh token a second time.

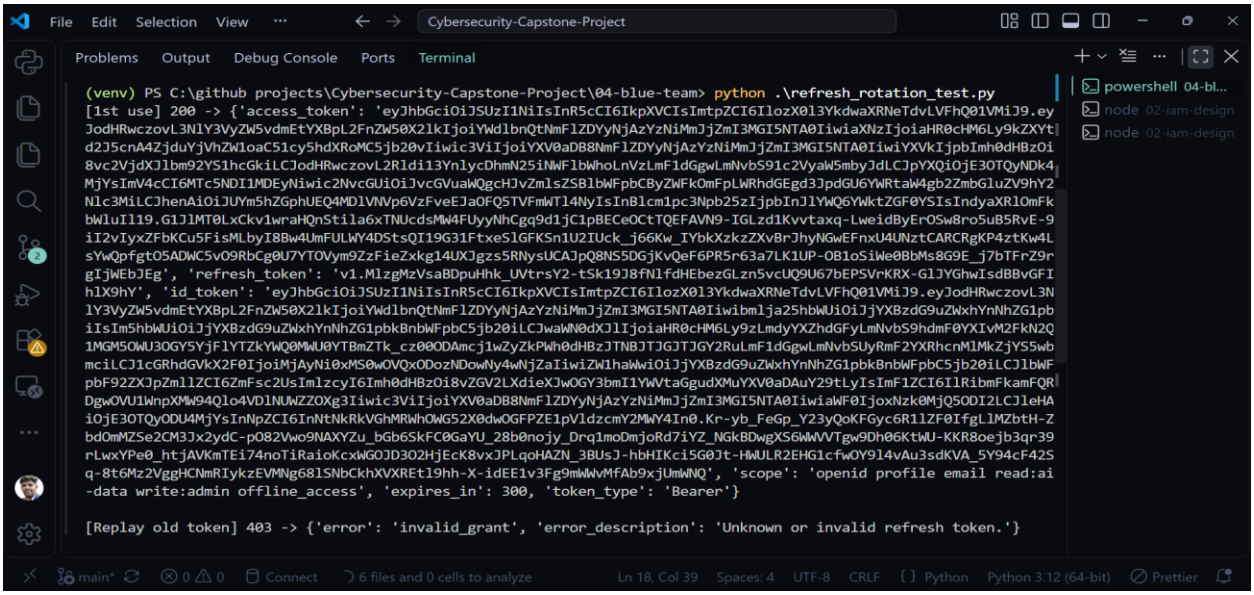


Figure 9: *Terminal showing rejection of the previously rotated refresh token.*

4. Anomaly Detection

I wrote a Python detection script that monitors three conditions: an LLM API call volume spike (more than 20 requests in 60 seconds), a scope change between consecutive requests from the same agent, and reuse of a token after its recorded expiry.

4.1 API Request Volume Spike

Trigger an alert when an agent makes more than 20 requests within 60 seconds.

4.2 Scope Change

Detect unexpected changes in permissions or scopes between consecutive requests from the same agent.

4.3 Expired Token Reuse

Detect attempts to reuse a token after its expiry has been recorded.

I replayed the Project 3 attack patterns through the detector and confirmed each alert fires with a timestamp, the agent identity, and the event type.

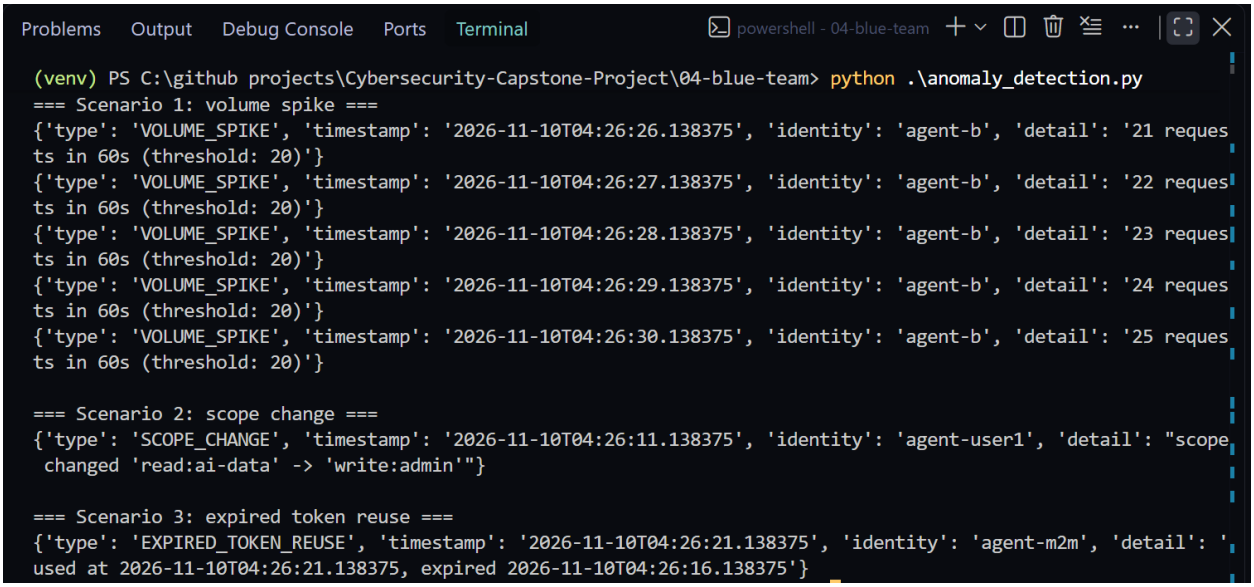


Figure 10: *Terminal showing anomaly alerts with timestamp, identity, and event type.*

5. Before/After Security Comparison

I re-ran all five testable Project 3 attacks against the hardened application and recorded the outcome and the specific control responsible for blocking each one.



Figure 11: *Project 3 versus Project 4 comparison table showing all five attacks and percentage improvement.*

Conclusion

Project 4 implements multiple defensive controls to address the AI identity security weaknesses identified during Project 3.

Input and output guardrails reduce the risk of prompt manipulation and credential disclosure. Ed25519 signatures strengthen agent identity verification. Auth0 hardening reduces token exposure and improves refresh-token security. Anomaly detection identifies suspicious behavior before it develops into a larger security incident.

The project demonstrates the importance of combining AI guardrails, cryptographic identity, secure token management, authorization, and continuous monitoring.

---------- **END OF PROJECT 4** ----------